EduGenie Project Documentation

1. Introduction

- **Project Title:** EduGenie — Google Gemini Powered Learning Assistant
- **Team Leader:** Padmavathi
- **Date:** 25/09/2026

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through generative AI. It enables students of all academic levels to ask questions and receive smart, concise answers, understand complex concepts through simplified explanations, generate quizzes from topics or text, receive personalized learning recommendations, and summarize large educational passages. Built with FastAPI for the backend and a simple HTML + CSS frontend, EduGenie leverages lightweight local models and cloud-based AI for efficiency and accessibility, working well even on devices like the Mac M1.

2. Project Overview

EduGenie was developed to address common challenges in education such as lack of personalized learning tools and limited access to immediate explanations. The objectives are:

- Provide smart, concise answers to student questions.
- Offer simplified explanations of complex concepts in an easy-to-understand manner.
- Generate multiple-choice quizzes for self-assessment and practice.
- Summarize long educational passages for quick revision.
- Deliver personalized learning paths with structured topics, timelines, and resources.
- Provide a user-friendly web interface accessible to students of all levels.

This project is **not subject-specific**, making it adaptable for various domains of learning — from science and math to programming and languages.

3. System Architecture

The architecture of EduGenie consists of several interconnected layers:

1. **Input Layer:** Accepts user inputs through an HTML form with a task dropdown (Explain, QnA, Quiz, Summary, Recommend Path) and text area.
2. **Frontend Layer:** Simple, responsive HTML + CSS interface with a submit button.
3. **Backend Layer (FastAPI):** Routes each request to the respective endpoint based on the user's selection.
4. **Model Layer:**

- **LaMini-Flan-T5-783M (local):** Used for concept explanation — instruction-tuned, lightweight, CPU-compatible.
- **Google Gemini 1.5 Pro (cloud API):** Used for Q&A, summarization, quiz generation, and learning paths — advanced reasoning, structured outputs, cloud inference.

5. **Application Logic:** Implements functions like `explain_topic`, `answer_question_with_gemini`, `generate_quiz`, `summarize_text`, and `get_learning_recommendations`.
6. **Execution Layer:** Managed via Uvicorn ASGI server with memory optimization for both CPU and GPU inference.

Architectural Flow

```
Start (User opens EduGenie)
      ↓
Frontend: User Input
      ↓
FastAPI Backend: Route to respective endpoint
      ↓
┌──────────────┬──────────┬──────────┬──────────┬──────────┐
│ Explanation  │   Q&A    │   Quiz   │ Summary  │ Learning │
│ /explain     │ /qa      │ /quiz    │/summarize│/learn/recomm│
└──────────────┴──────────┴──────────┴──────────┴──────────┘
      ↓
Frontend: Display Results
      ↓
End (Results shown, user interacts again)
```

This layered approach makes the system modular and scalable for future upgrades.

4. Setup Instructions

To install and run EduGenie, follow these steps:

1. **Clone or download the repository** to your local machine.
2. **Create and activate a virtual environment:**
   ```bash
   python -m venv .venv
   # Windows:
   .venv\Scripts\Activate.ps1
   # macOS / Linux:
   ```

source .venv/bin/activate
```
3. **Install required libraries:**
```bash
pip install fastapi uvicorn jinja2 python-multipart python-dotenv google-genai transformers torch
```
4. **Obtain a Google Gemini API Key:**
- Visit https://aistudio.google.com/app/apikey
- Sign in with your Google account
- Create a new project and enable the Gemini API
- Generate an API key and store it securely in a `.env` file:
```
GEMINI_API_KEY=your_actual_key_here
```
5. **(Optional) Ensure GPU with CUDA** is available for faster local model inference.
6. **Run the server:**
```bash
uvicorn main:app --reload
```
7. **Open the app** in your browser at:
```
http://127.0.0.1:8000
```

Pre-requisites

Tool	Version	Purpose
Python	3.10+	Core runtime
FastAPI	Latest	Backend framework
Uvicorn	Latest	ASGI server
Jinja2	Latest	HTML templating
Google Gemini API Key	—	Cloud AI model access
HTML & CSS	—	Frontend (in `/templates` and `/static`)

---

5. Folder Structure

A typical folder structure is as follows:

```
EduGenie/
├── main.py                    # FastAPI app + all endpoints
```
```

```
├── explanation_module.py        # Concept explanation logic (LaMini)
├── qna.py                       # Question answering (Gemini)
├── quiz_module.py               # Quiz generation (Gemini)
├── summary_module.py            # Summarization (Gemini)
├── learning_path.py             # Learning recommendations (Gemini)
├── config.py                    # Configuration & API initialization
├── requirements.txt             # Python dependencies
├── .env                         # API key (not committed)
├── .gitignore                   # Files to ignore
├── README.md                    # Basic project info
├── templates/
│   └── index.html               # HTML frontend
├── static/
│   └── style.css                # Styling
├── docs/                        # Documentation and reports
│   └── EduGenie_documentation.docx
└── tests/                       # Unit tests for functions
    └── test_modules.py
```

This ensures clarity and maintainability for contributors and developers.

6. Running the Code

Running the project involves:

1. Loading the pretrained local model (LaMini-Flan-T5-783M) and tokenizer on first `/explain` call.
2. Initializing the FastAPI app with all five REST endpoints.
3. Serving the Jinja2-templated `index.html` at the root route.
4. Starting the app locally at **http://127.0.0.1:8000**.
5. Accepting user inputs via the web interface and returning AI-generated responses in real time.

API Endpoints

Method	Endpoint	Description
GET	`/`	Frontend UI
GET	`/qa?question=...`	Question answering
POST	`/explain`	Explain a topic
POST	`/summarize`	Summarize text
POST	`/quiz`	Generate 3 MCQs
GET	`/learn/recommendations?topic=...`	Learning path

| GET | `/health` | Health check |

This makes deployment seamless for testing and demonstrations.

7. API Documentation

`answer_question_with_gemini(question: str) -> str`
- **Purpose:** Answers a user's question using Google Gemini.
- **Parameters:** `question` — the user's question string.
- **Returns:** A concise, accurate answer as a string.
- **Endpoint:** `GET /qa`

`explain_topic(topic: str) -> str`
- **Purpose:** Provides a simplified, beginner-friendly explanation of a concept.
- **Parameters:** `topic` — concept or topic to explain.
- **Returns:** Explanatory text as a string.
- **Endpoint:** `POST /explain`

`generate_quiz(text: str) -> list`
- **Purpose:** Creates 3 multiple-choice questions from a passage.
- **Parameters:** `text` — the source passage.
- **Returns:** A Python list of dicts, each with `question`, `options`, and `answer`.
- **Endpoint:** `POST /quiz`

`summarize_text(text: str) -> str`
- **Purpose:** Summarizes long passages into concise versions.
- **Parameters:** `text` — the passage to summarize.
- **Returns:** A short summary string.
- **Endpoint:** `POST /summarize`

`get_learning_recommendations(topic: str) -> str`
- **Purpose:** Generates a personalized learning path (beginner → intermediate → advanced).
- **Parameters:** `topic` — the subject the student wants to learn.
- **Returns:** A structured learning plan with resources.
- **Endpoint:** `GET /learn/recommendations`

8.. User Interface

The interface is designed with a clean, responsive HTML + CSS layout organized as five interactive cards:

- **Concept Explanation:** Input a topic, receive a detailed breakdown.
- **Q&A:** Ask any question, get a direct answer.

- **Quiz Generator:** Input a passage, receive 3 MCQs with answers.
- **Summarization:** Paste long text, receive a concise summary.
- **Learning Recommendations:** Enter a topic, get a structured learning path.

Its simplicity ensures accessibility without technical barriers — usable by school students, self-learners, and educators alike.

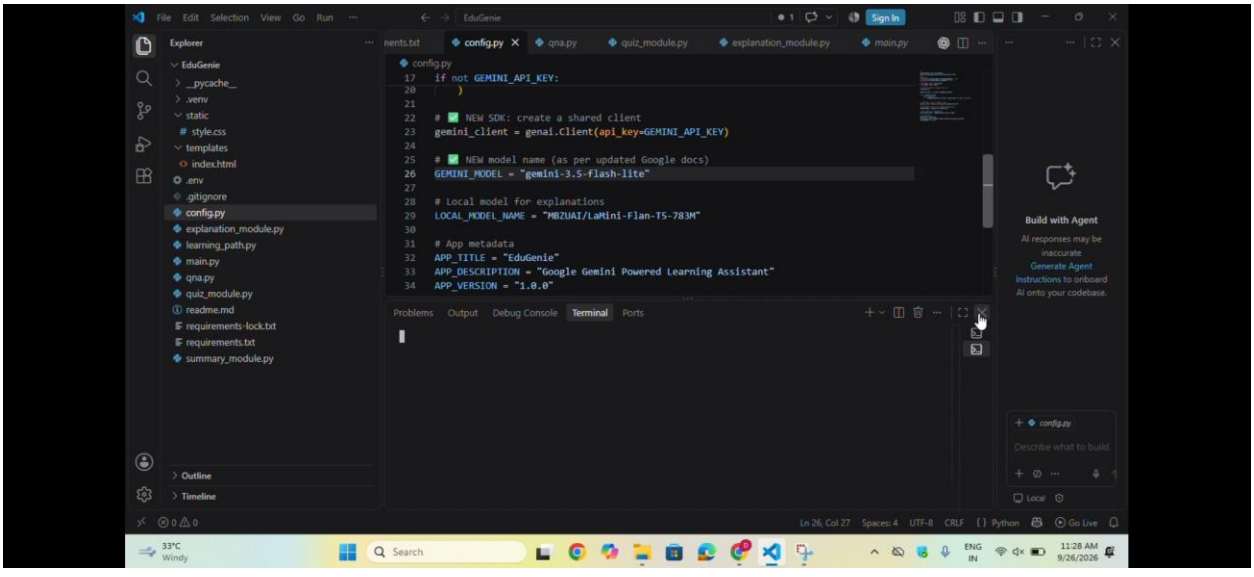
10. Conclusion

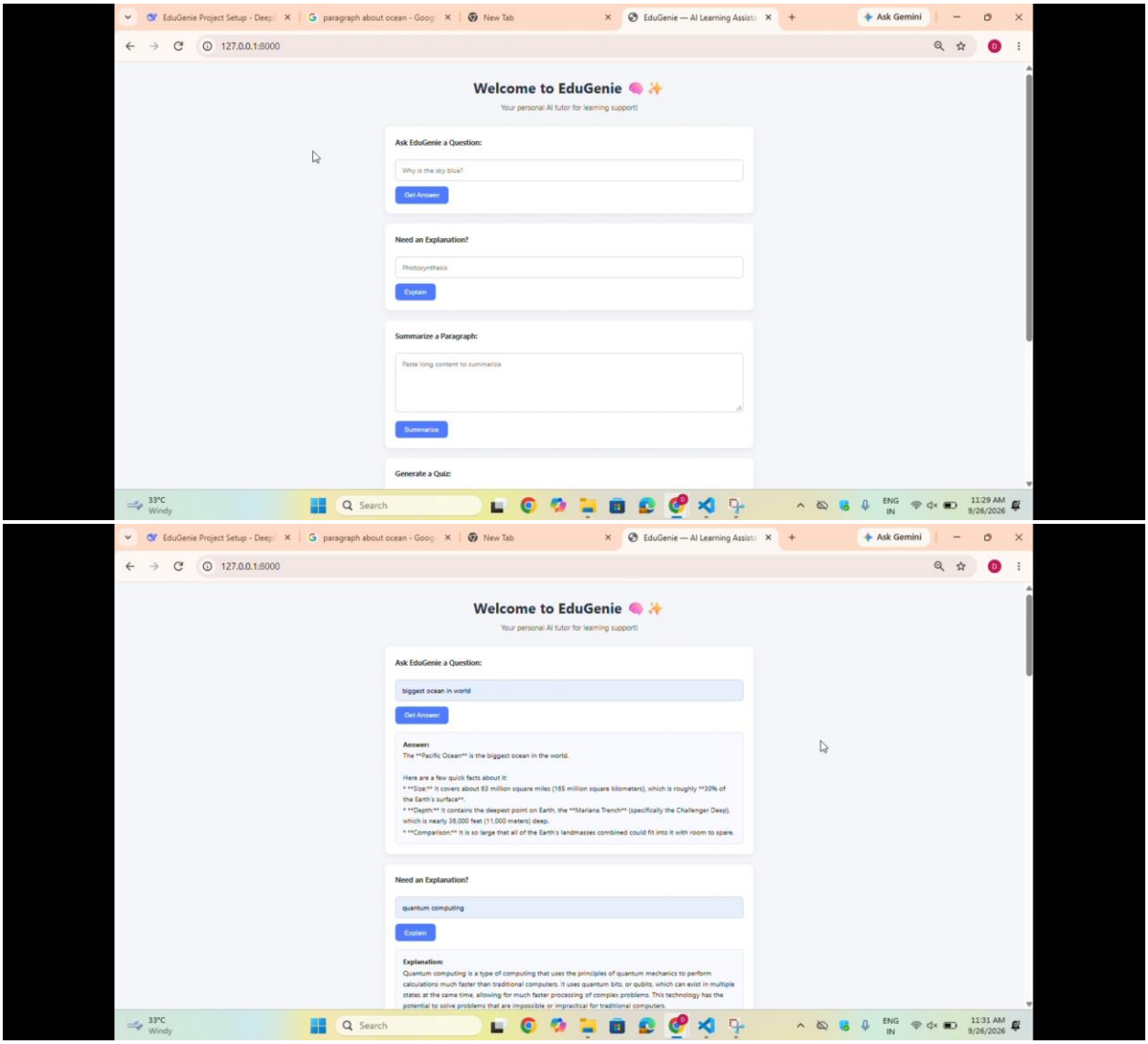
EduGenie is a robust demonstration of integrating AI into education. It is scalable and adaptable, opening doors for advanced features such as:

- Multi-language support (Tamil, Hindi, English, etc.).
- Tracking student progress with analytics dashboards.
- Custom difficulty levels for quizzes (easy, medium, hard).
- Voice-based interaction (speech-to-text and text-to-speech).
- Integration into mobile or LMS platforms.
- Adaptive learning paths powered by data analytics.
- Group study sessions and teacher/parent dashboards.

Through its modular architecture, cloud-backed intelligence, and lightweight local inference, EduGenie redefines the learning experience for digital natives and underserved communities alike — poised to become a comprehensive learning companion for the modern era: smart, scalable, and learner-first.

11. Screenshots





11. Future Enhancements

Here are some future enhancements that could be implemented for EduGenie to make it more powerful and practical:

1. Personalized Learning
- Adaptive difficulty: adjust quiz complexity based on student performance.
- User profiles storing previously learned concepts and quizzes attempted.
- AI-driven recommendations for "next topics to learn".

2. Enhanced Quiz Features
- Question banks for different difficulty levels (easy, medium, hard).
- Support for images, diagrams, and mathematical equations in questions.
- Automatic grading and performance analytics.

3. Multi-Language Support
- Explanations and quizzes in multiple languages (English, Tamil, Hindi, etc.) using multilingual models.
- Useful for global adoption and regional learning.

4. Voice Interaction
- Speech-to-text for voice-based questions.
- Text-to-speech for concept explanations so students can listen while studying.

5. Integration with External Platforms
- Integration with LMS platforms like Moodle, Google Classroom, or Canvas.
- Export quizzes and results to PDF/Excel for teachers.
- API endpoints for integration with mobile or third-party apps.

6. Offline & Mobile Support
- Mobile app version (Android / iOS).
- Allow downloading of concept explanations and quizzes for offline usage.